OpenHoopScore 项目规划方案

一、项目基本信息（GitHub 仓库核心内容）

1. 项目介绍

OpenHoopScore 是**面向基层篮球赛事**的开源轻量级计分与数据管理系统，聚焦「校园比赛、业余联赛、爱好者赛事」场景，为赛事组织者、裁判、球队提供**免费、易用、高效**的数据管理工具。

核心解决痛点：传统纸质计分易出错、数据难追溯；现有商业系统门槛高/收费贵，基层赛事缺乏轻量化工具。

2. 核心功能（MVP 最小可行版本）

模块	功能描述
赛事管理	创建/编辑赛事（名称、时间、地点、参赛队）、赛程编排、赛事状态（进行中/已结束）
实时计分	裁判端：得分（1/2/3 分）、犯规、暂停、节次切换；观众端：实时比分、事件流同步
数据统计	球队总得分、球员得分/犯规/篮板等基础统计，支持赛后导出（CSV/PDF）
用户与权限	角色区分：管理员（赛事创建）、裁判（计分操作）、观众（只读查看）

3. 技术栈选择（推荐全栈方案，兼顾易用性与扩展性）

- **前端**：React + Vite + TypeScript + TailwindCSS（快速构建响应式界面，适合实时交互场景）
- **后端**：Node.js + Express + MongoDB（轻量级 REST API，MongoDB 灵活存储赛事/球员/事件数据）
- **实时通信**：Socket.io（实现裁判端操作→观众端实时同步）
- **部署**：Vercel（前端）+ Railway（后端+数据库），或 Docker 本地部署

二、制作前的准备工作

1. 技术储备

- 前端：熟悉 React 组件化开发、TypeScript 类型系统、TailwindCSS 样式工具。
- 后端：掌握 Node.js 异步编程、Express 路由/中间件、MongoDB 文档模型设计。
- 实时通信：理解 Socket.io 服务端/客户端通信逻辑（如 `emit`/`on` 事件绑定）。

2. 环境搭建

- 安装 Node.js（≥18.x）、npm/yarn/pnpm 包管理器、Git 版本控制工具。
- 注册 GitHub 账号，配置 SSH 密钥（用于本地仓库与远程仓库关联）。

3. 需求细化与设计

用户故事梳理：

- 作为赛事组织者，我需要创建赛事并邀请球队，以便管理赛程。
- 作为裁判，我需要实时记录得分/犯规，以便同步给观众。

- 作为观众，我需要查看实时比分和赛后统计，以便跟踪赛事。

数据库设计（MongoDB 集合示例）：

- `events`（赛事）：`{ _id, name, date, location, teams: [teamId], status }`
- `teams`（球队）：`{ _id, name, eventId, players: [playerId] }`
- `players`（球员）：`{ _id, name, teamId, stats: { points, fouls, ... } }`
- `gameEvents`（实时事件）：`{ _id, eventId, type: 'score'/'foul', teamId, playerId, timestamp }`

三、项目开发流程（从 0 到 1 落地）

1. 初始化 Git 仓库

```
mkdir OpenHoopScore && cd OpenHoopScore
git init
# 创建 .gitignore（排除 node_modules、.env 等敏感文件）
echo "node_modules/\n.env" > .gitignore
git add . && git commit -m "init repo"
```

2. 前端开发（以 React + Vite 为例）

- 初始化项目：`npm create vite@latest frontend -- --template react-ts`
- 安装依赖：`cd frontend && npm install tailwindcss postcss autoprefixer socket.io-client axios`
- 配置 TailwindCSS：执行 `npx tailwindcss init -p`，修改 `tailwind.config.js` 包含 `./src/**/*.{js,ts,jsx,tsx}`。
- 核心页面开发：
  - 赛事列表页：展示所有赛事，支持筛选/搜索。
  - 赛事详情页：集成「裁判计分面板」+「观众实时视图」（通过 Socket.io 同步数据）。

3. 后端开发（以 Node.js + Express 为例）

- 初始化项目：`mkdir backend && cd backend && npm init -y`
- 安装依赖：`npm install express mongoose socket.io dotenv cors`
- 启动服务：编写 `server.js`，集成 Express 路由、MongoDB 连接、Socket.io 实例。
- 核心接口开发：
  - 赛事 CRUD：`POST /api/events`（创建）、`GET /api/events`（列表）。
  - 实时事件推送：Socket.io 监听 `scoreUpdate` 事件，向所有连接的客户端广播新比分。

4. 测试与部署

- 本地测试：用 Postman 测试后端 API，用浏览器测试前端页面，验证 Socket.io 实时同步。
- 部署上线：
  - 前端：推送到 Vercel（自动识别 Vite 项目，一键部署）。
  - 后端：推送到 Railway（自动识别 Node.js 项目，配置 MongoDB 环境变量）。

四、CodeBuddy 输入指令方案（AI 辅助开发示例）

CodeBuddy 可通过自然语言指令生成代码/设计方案，以下是关键阶段的指令模板：

1. 需求分析阶段

“帮我设计 OpenHoopScore 的核心功能模块，要求覆盖赛事管理、实时计分、数据统计，输出功能清单和用户故事。”

2. 数据库设计阶段

“用 MongoDB 设计 OpenHoopScore 的数据库结构，需要存储赛事、球队、球员、实时事件，给出集合字段定义和关联关系。”

3. 前端组件开发阶段

“用 React + TypeScript + TailwindCSS 写一个「裁判计分面板」组件，支持记录 2 分球、3 分球、犯规，点击后通过 Socket.io 发送事件到后端。”

4. 后端接口开发阶段

“用 Node.js + Express + Mongoose 写一个创建赛事的 API 接口，接收 {name, date, location, teams} 参数，存入 MongoDB 的 events 集合，返回成功响应。”

5. 实时通信调试阶段

“帮我写 Socket.io 的服务端代码，当收到客户端的 ‘scoreUpdate’ 事件时，广播最新的比分数据给所有连接的观众端。”

五、GitHub 仓库完善建议

- **README.md**：清晰说明项目背景、功能、技术栈、本地运行步骤（如 `cd frontend && npm run dev`）。
- **LICENSE**：添加开源协议（如 MIT），明确使用权限。
- **Issues/Projects**：用 Issues 收集反馈，用 Projects 看板管理开发进度（如「待办」「进行中」「已完成」）。

通过以上步骤，可系统化完成 OpenHoopScore 从规划到上线的全流程。